CH
09

Caching

Caffeine · Redis · HTTP ETag · CDN · Strategies

Overview

Multi-layered caching: Caffeine (JVM heap), Redis (distributed), HTTP ETags, Nginx proxy cache, and CDN edge caching. Spring Cache abstraction unifies Caffeine and Redis.

Cache Hierarchy

- **CDN / Edge (CloudFront) – product images, Cache-Control: m**
  - Nginx Proxy Cache – /api/v1/products; proxy_cache_valid 200
    - HTTP ETag – ShallowEtagHeaderFilter; 304 Not Modified on ve
      - Spring Cache (@Cacheable, @CacheEvict, @CachePut)
        - Redis – RedisCacheManager (products: 5min, categories: 1h
        - Caffeine – maximumSize(10 000) expireAfterWrite(5 min) r
          - Hibernate L2 (Caffeine via jcache) – @Cache(READ_WRITE) o

Caching Strategies

Strategy	Class	Mechanism
Cache-aside	ProductService.findById()	Check cache → miss → DB → populate cache
Write-through	ProductService.update()	Update DB and cache in same transaction
Write-behind	ProductViewCountService	Increment Redis counter; @Scheduled flush to DB every 5 min
Refresh-ahead	CategoryCacheWarmer	@EventListener(ApplicationReadyEvent) preloads all categories

Cache Eviction Policies

Level	Policy	Config
Caffeine	LRU eviction	maximumSize(10_000) → evicts LRU when full
Redis	allkeys-lru	maxmemory-policy in docker-compose.yml Redis args
Stampede prevention	Redisson RLock	getLock("product-cache-init") → only one instance rebuilds cache

Cache Provider Comparison

Feature	Caffeine	Redis	Hazelcast	Memcached
Location	JVM Heap	External process	Distributed in-process	External process
Data structures	None (K-V only)	String, Hash, List, Set, ZSet, Stream	Map, Set, Queue, Topic	K-V only
Clustering	No	Yes (Redis Cluster)	Yes (native)	No

Feature	Caffeine	Redis	Hazelcast	Memcached
Persistence	No	RDB + AOF	Yes	No
TTL support	Yes (expireAfterWrite)	Yes (EX / EXPIRE)	Yes	Yes
Spring Boot auto-config	Yes	Yes	Yes	Manual
ShopVerse usage	L1 heap cache	Primary distributed cache	@ConditionalOnProperty	@ConditionalOnProperty